

Handwritten Digit Recognition System

Name: Jhenkar P

College: Bangalore Institute of technology

Branch: AI & ML

1. Introduction

- Reading handwritten digits is a classic problem in machine learning. People use it for things like reading postal codes, bank cheques and forms. For this project I trained a neural network to recognise the digits 0 to 9 and then built a web app around it, so that anyone can try it without touching any code.
- The app can do two things. You can draw a single digit on the screen, or you can upload a photo of a page with several handwritten digits and it will read all of them.

2. Objective

- Train a model that recognises handwritten digits.
- Build a web page where a user can draw a digit and see the prediction.
- Let the user upload a photo with many digits and show every digit that was found
- Make it work on photos of normal lined notebook paper, not only on clean images.
- How to run it - The setup steps are in `README.md`.

3. Tools used

Part	Tool
Language	Python
Model training	TensorFlow / Keras
Web page	Streamlit, streamlit-drawable-canvas
Server	FastAPI, Uvicorn
Image processing	OpenCV, NumPy
Charts and tables	Pandas
Editor	VS Code

4. Dataset and model

- I used the MNIST dataset. It has 70,000 grayscale images of handwritten digits, each 28 x 28 pixels. 60,000 are used for training and 10,000 for testing. In every image the digit is white on a black background and sits in the middle.
- After training I looked at the images the model got wrong. A few examples: a 7 read as a 2, a 9 read as a 4, a 5 read as a 3 and a 0 read as a 5.
- The trained model is saved as `digit\_model.keras`.
- The model accuracy is 98%.

5. System design

The project has two parts that talk to each other over HTTP.

- **Streamlit page (`app.py`)**- This is what the user sees. It has two tabs: one with a drawing canvas and one for uploading an image. It sends the picture to the server and shows the result.

- **FastAPI server** (`backend.py`)- It loads the model once when it starts. It has three endpoints:

- `predict` for a single drawn digit
- `predict-image` for a photo with many digits

I split it this way so the model code and the web page are separate. The server can be reused by some other app later, and the page does not need TensorFlow loaded inside it.

6. Getting an image ready for the model

The model was trained on MNIST, where every digit is centered and fits inside a 20 x 20 box in the middle of the 28 x 28 image. A digit drawn by a user does not look like that. It can be big or small and in any corner of the canvas. If I just shrink the whole picture to 28 x 28, the predictions are poor.

So the server does what MNIST did:

1. Crop the picture tight around the digit.
2. Scale it so the longest side is 20 pixels, keeping the shape.
3. Place it on a black 28 x 28 image.
4. Move it so its center of mass is in the middle.

This one step made a big difference for hand-drawn input.

7. Reading many digits from a photo

A photo of a notebook page is much messier than a canvas drawing. The server handles it in these steps:

1. Resize the photo so the longest side is 1000 pixels.
2. Black and white. An adaptive threshold is used instead of one fixed cut-off, because the light is rarely even across a page and one side is often darker.
3. Remove the ruled lines. Lines on the paper are long and thin. I find them by looking for very long horizontal and vertical strokes and then erase them. Where a digit crossed a line, the stroke gets a small gap, so I close those gaps afterwards.
4. Find the shapes. Each connected shape gets a box around it.
5. Throw away what is not a digit. Boxes are removed if they are too small, too long and thin, touching the edge of the picture, or far from the typical digit height.
6. Sort into rows. Boxes are grouped by height on the page, then ordered left to right.
7. Predict. Each box goes through the same preparation as in section 6 and then into the model. The results are shown on the picture with a box and a label on each digit.

8. Problems I ran into.

Converting the model for the browser- My first plan was to convert the model with TensorFlow.js and run it directly in the browser. The converter kept failing because of package conflicts (protobuf, `tf\_keras`, and several modules it expected such as `jax` and `tensorflow\_decision\_forests`). I did get the conversion to finish in the end, but it needed workarounds that I did not like. I switched to a Python server with FastAPI, which is simpler and cleaner.

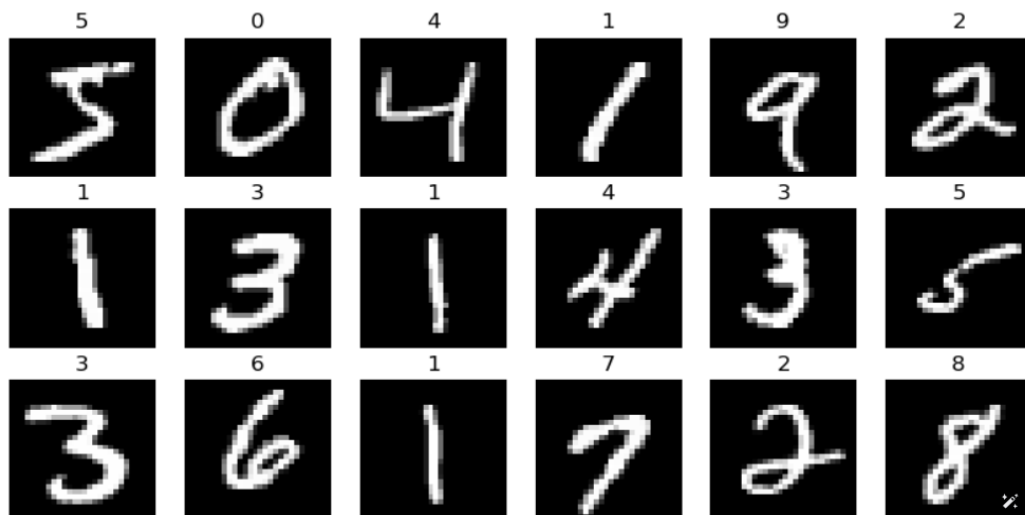
Package versions- TensorFlow 2.16 does not work with NumPy 2, and a newer protobuf broke model loading. Installing OpenCV pulled in NumPy 2 and stopped TensorFlow from importing. I fixed it by using a fresh virtual environment and pinning the versions in `requirements.txt`.

Ruled lines read as digits- My first version of the photo reader found 17 "digits" on a notebook page. Almost all of them were pieces of the ruled lines, and several real digits were missed because they were attached to a line. Adding the line removal step in section 7 fixed this. On a test photo of ten handwritten digits, the new version found exactly ten shapes and ignored the lines, the page border and the printed logo.

Uneven light. A single threshold value did not work on photos with a shadow. The adaptive threshold solved this.

9. Testing and results

```
In [3]: fig, axes = plt.subplots(3, 6, figsize=(10, 5))
        for i, ax in enumerate(axes.flat):
            ax.imshow(x_train[i], cmap='gray')
            ax.set_title(y_train[i])
            ax.axis('off')
        plt.show()
```



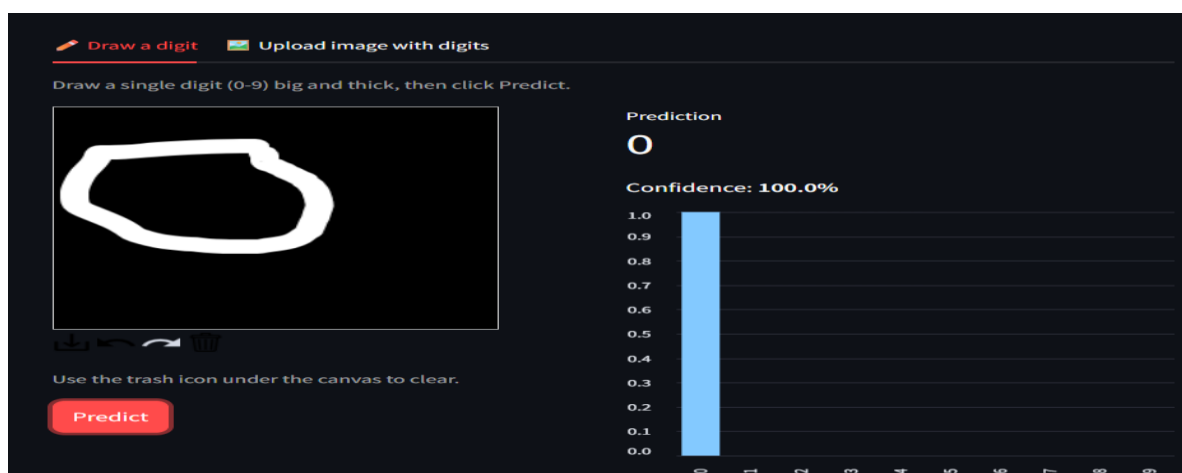
```
# add channel dimension and scale to 0-1
x_train = x_train.reshape(-1, 28, 28, 1).astype('float32') / 255
x_test = x_test.reshape(-1, 28, 28, 1).astype('float32') / 255

# keep last 5000 training images aside for validation
x_val, y_val = x_train[-5000:], y_train[-5000:]
x_train, y_train = x_train[:-5000], y_train[:-5000]

print('train:', x_train.shape[0])
print('val:', x_val.shape[0])
print('test:', x_test.shape[0])
```

```
train: 55000
val: 5000
test: 10000
```

```
Epoch 1/30
430/430 ————— 47s 108ms/step - accuracy: 0.9792 - loss: 0.0767 - val_accuracy: 0.9942 - val_loss: 0.0305 - learn
ing_rate: 1.2500e-04
Epoch 2/30
430/430 ————— 50s 117ms/step - accuracy: 0.9805 - loss: 0.0705 - val_accuracy: 0.9940 - val_loss: 0.0318 - learn
ing_rate: 1.2500e-04
Epoch 3/30
430/430 ————— 50s 115ms/step - accuracy: 0.9827 - loss: 0.0655 - val_accuracy: 0.9920 - val_loss: 0.0376 - learn
ing_rate: 1.2500e-04
Epoch 4/30
430/430 ————— 52s 121ms/step - accuracy: 0.9830 - loss: 0.0591 - val_accuracy: 0.9948 - val_loss: 0.0287 - learn
ing_rate: 6.2500e-05
Epoch 5/30
430/430 ————— 50s 116ms/step - accuracy: 0.9831 - loss: 0.0601 - val_accuracy: 0.9946 - val_loss: 0.0324 - learn
ing_rate: 6.2500e-05
Epoch 6/30
430/430 ————— 56s 130ms/step - accuracy: 0.9821 - loss: 0.0637 - val_accuracy: 0.9942 - val_loss: 0.0331 - learn
ing_rate: 6.2500e-05
Epoch 7/30
430/430 ————— 56s 129ms/step - accuracy: 0.9835 - loss: 0.0583 - val_accuracy: 0.9948 - val_loss: 0.0308 - learn
ing_rate: 3.1250e-05
Epoch 8/30
430/430 ————— 52s 119ms/step - accuracy: 0.9817 - loss: 0.0614 - val_accuracy: 0.9946 - val_loss: 0.0303 - learn
ing_rate: 3.1250e-05
Epoch 9/30
430/430 ————— 52s 122ms/step - accuracy: 0.9837 - loss: 0.0600 - val_accuracy: 0.9950 - val_loss: 0.0297 - learn
ing_rate: 1.5625e-05
```



 Draw a digit  Upload image with digits

Upload a photo or scan with several handwritten digits. Best results: dark ink on plain light paper, good lighting, digits not touching.

Choose an image



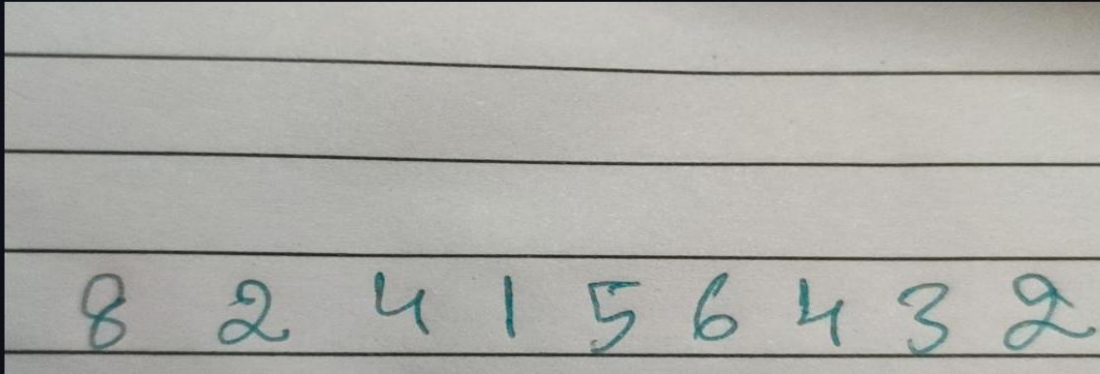
Drag and drop file here

Limit 200MB per file • PNG, JPG, JPEG

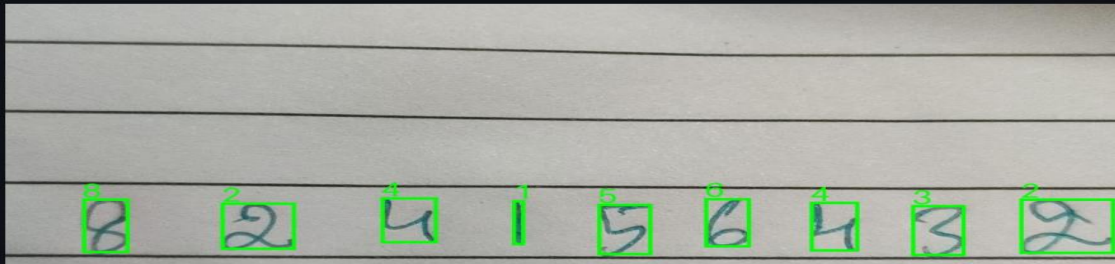
Browse files



dd.jpeg 146.4KB

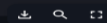


Recognize digits



Found 9 digit(s)

824156432



#	digit	confidence %
1	8	100
2	2	100
3	4	100
4	1	100
5	5	100
6	6	100
7	4	100
8	3	100
9	2	100

10. Limitations

- The model only knows MNIST-style handwriting, so unusual styles are sometimes read wrong. An open-top 9 that looks like a q is one example.
- Digits that touch each other are treated as a single shape and will not be read properly.
- A photo taken at a strong angle can break the line removal.
- It only recognises digits, not letters or printed text.

11. Future work

- Train on more varied handwriting, or add extra training images with different pens and paper.
- Split touching digits automatically.
- Straighten tilted photos before reading them.
- Read whole numbers as one value, such as phone numbers or amounts.
- Put the app online so others can use it without installing anything.

Conclusion

The project works as a complete app. A user can draw a digit or upload a photo of handwriting and get the digits back with a confidence score. The biggest improvements came from preparing the images properly, in the same way MNIST was prepared, and from cleaning the notebook lines out of photos before reading them. The main weakness is that the model has only seen MNIST, so it makes mistakes on handwriting that looks very different from that.